Software Testing ETSN20

Projects in Software Testing

Per Runeson
with contributions from Carina Andersson, Elizabeth Bjarnason, Markus Borg, Emelie Engström, Hussan Munir, Mika Mäntylä, Qunying Song, Sergio Rico, Sule Tekkesinoglu and Thomas Thelin, Lund University, Dept. Computer Science
October 26, 2026

Contents

1 Introduction 2
2 Learning Objectives 2
3 Activities 2
4 Using AI tools 3
4.1 Limitations and risks of AI use 3
4.2 Guidelines for AI use 3
5 Schedule 3
6 Assessment 4
7 Report 4
8 Presentation 5
9 Practical Project Areas 5
9.1 Evaluate tools for software testing or review 5
9.2 Evaluate test methods 5
9.3 Develop an automated test suite 5
10 Research Project Areas 6
10.1 How is testing performed for Agile processes? 6
10.2 Can test cases be generated automatically from the code? 6
10.3 How to use LLMs to generate (good) tests? 6
10.4 How to test machine learning systems? 7
10.5 How to test interpretability in machine learning system? 7
10.6 What are the minimal test practices needed in a start-up company? 8
10.7 Is it more efficient to generate test cases from a model? 8
10.8 How do you test for security? 8
10.9 Do static code analysis tools help improve software code quality? 8
10.10 How do you know that your test cases are effective? 9
10.11 Flaky or non-deterministic tests 9
10.12 How to test the energy consumption of software systems? 9
10.13 How organize assertions in test cases? 10

1 Introduction

This document gives the practical details regarding the project in the course Software Testing. The project in the course is equivalent to 2 weeks of full-time studies (3 ECTS credit points). The main objective of the project is to gain a deeper understanding of a specific area within verification and validation (V&V) of software products. A project group consists of 4 (±1) persons, who perform the project together. All project members should be involved, and the total effort should be evenly distributed among participants.

2 Learning Objectives

The objective of the project is to learn a specific area of software testing. Furthermore, the structure of the project makes it possible to learn important parts that will be practiced later in in the education. Hence, the main learning points of the project are:

- Learn a specific area of software testing
- Collect and summarize research information
- Critical thinking beyond the written information
- Present information in a structured way

There are two types of projects to choose among:

- **Practical project:** Evaluate a testing technique/tool or use a testing technique to verify a system or design. This entails applying a testing technique or a testing or analysis tool.
- **Research project:** Solve a research problem; survey the state-of-the-art and identify the research problems in some area; develop and justify an extension to an existing technique; etc.

3 Activities

The main activities in the project are:

1. **Decide project type and subject**, i.e. choose to perform a *research project* or a *practical project*, then select a specific subject from the list below. Only one group may choose a specific subject. Other subjects may be chosen but must be agreed with your project supervisor.
2. **Establish a collaboration agreement**. Register your group in Canvas and establish a collaboration agreement, including, a) ambition level within the group (pass/pass with distinction), b) communication format and working platform, c) meeting and work schedule, d) task breakdown and division.
3. **Find relevant literature**. In each subject area, there is a suggestion of research reports to start reading. Other research literature can be found in the Finn database http://www.lub.lu.se. Systematic literature reviews (SLR) or mapping studies (MS) are good sources of synthesized literature. For research projects, 7-10 references to related research literature should be chosen. For practical projects, somewhat fewer research references are needed.
4. **Outline the report**. Specify heading levels 1 and 2, write in short sentences or bullet items the key message of each section. Start using the correct template directly, see below. The more you write in the outline, the better feedback you will get. Submit in Canvas.
5. **Consult the supervisor**. During reading and writing, you must consult the supervisor on one occasion. Book meeting time according to course schedule deadline.
6. **Read and perform practical work**. Read and understand the new area of software testing and, if a practical project has been chosen, perform practical work.
7. **Write the report**. The report is to be written in English. The report shall be written in the IEEEtran Proceedings Format[1]. Submit in Canvas.
8. **Present the report**. Communicate the area to other people who have knowledge of software testing, but not of the specific area. This is done during the course conference. Each group will be given a time slot in which to present their work. The presentation shall be in English.

[1] For Word: https://www.conference-publishing.com/templates/MSW_USltr_format.doc . For LaTeX: use the LaTeX class file IEEEtran v 1.8 (http://www.ctan.org/tex-archive/macros/latex/contrib/IEEEtran) and the following configuration (without option 'compsoc' or 'compsocconf'): \documentclass[conference]{IEEEtran}or use the Overleaf template https://www.overleaf.com/project/5f9864a592bfa20001211908

4 Using AI tools

AI tools, such as ChatGPT, Copilot and Elicit can be used to support the work in the project, but not to "solve" the task as such. Such tools can, for example, be used to:

- Summarize research articles and identify relevant contributions
- Generate examples of research questions and methodological approaches
- Create sample data to illustrate data analysis
- Support language review and structure suggestions in report writing
- Train argumentation by generating alternative perspectives

4.1 Limitations and risks of AI use

AI can be a powerful tool, but there are also limitations and risks that you should be aware of.

- *Reduced learning through shortcuts.* If you use AI to generate texts or analyses without reflecting on yourself, it will limit your understanding of the subject – and miss the written exam where no assisting tools are permitted.
- *Errors and bias.* AI models can generate misleading or incorrect information, especially in complex research areas. You are responsible for ensuring that everything you claim is true and reasonable.
- *Insufficient source citation.* AI may generate text without providing clear sources, which makes it difficult to verify claims.
- *Unnecessary extra work.* It can sometimes take many iterations to get something useful out of these tools, especially in areas where you are not yet an expert yourself. Conversations within the group can be more time-efficient activities both in terms of learning and moving the project forward. Raw AI text is often less precise and of lower quality than text you formulate yourself.

4.2 Guidelines for AI use

If AI is used, it is important to:

- *The use is transparent.* You must report whether and how AI tools have been used. Discuss with your supervisor during the project and report all details in the final report's *Contribution statement*.
- *AI must not replace your own reasoning.* All content generated by AI must be critically reviewed, processed and adapted to the course's theories, models and concepts.
- *You must not mislead reviewers and examiners.* If AI-generated text, data or ideas have been used, this must be clearly stated. It is not allowed to present AI-generated material as your own. Misconduct in this respect will be reported.
- *Critical review required.* You must always double-check AI-generated information against reliable sources, especially when it comes to the course's learning objectives and content.

5 Schedule

Course week	Activities	Hand in to supervisor	Deadline
1	Create group and decide on a subject Search for literature	Decided subject for group	Fri Nov 6
2-3	Search literature Read literature Outline the report	Outline	Tue Nov 17
4-5	Read literature Write report Consult supervisor	Book time well in advance – mandatory!	Fri Nov 27
6	Write report	Report to supervisor	Fri Dec 11
7	Prepare presentation Course conference		Thu Dec 17
Exam week	Update report	If needed, update report to supervisor	Fri Jan 8

6 Assessment

The reports are scored on a scale fail–pass–pass with distinction (U, G, VG) on each of the following four characteristics:

- Form (0% of total marks):
  - Correct use of IEEE template (formatting of page, title, names, headings, paragraphs, table and figure captions, references, etc.)
  - Correct referencing style; also: each document listed in the reference section must be referenced from the text at least once
  - Language: no spelling/grammar errors, clarity of expression, appropriateness of expression (no slang!), no LLM chit-chat, correct usage of terminology
  - Observe page limit (5-7 pages, all inclusive)
- Work (40% of total marks):
  - Appropriate scoping (focus) of work
  - Sufficient depth of work (e.g., number, relevance, appropriateness of literature)
  - Appropriateness/correctness and level of detail of analysis
  - Discussion of limitations of the work done
  - No over-interpretation of findings
  - No unsubstantiated claims (i.e., claims must either be based on the results of the analysis or be based on results found in the (relevant) literature)
- Content (40% of total marks):
  - Structure in line with project description (see Section 6)
  - Correct flow of argument / consistency of argument
  - Analysis must relate to research problem
  - Discussion must be in the light of analysis results and research problem
- Presentation (20% of total marks):
  - Clarity of message
  - Contact with audience
  - Timeliness

The reports should be original work by the student groups and adhere to the university's guidelines on plagiarism[2] and the above instructions about AI tools. The project reports will be checked via Inspera Originality in Canvas[3].

You must achieve at least 80% of the maximum marks to receive the grade VG and 60% to achieve the grade G for your project.

7 Report

The report shall be written using the IEEE template (see footnote on page 2) and comprise 5-7 pages. For practical projects, details may be provided as appendixes. The report shall contain the following elements, although the section titles may be different, especially for practical projects:

- **Abstract**: summary of all parts in the report. The purpose of the abstract is to attract people to read your report.
- **Introduction**: introduce the chosen area. The purpose is to introduce the reader who is not familiar to the specific area, but knows software engineering and testing generally.
- **Description** of the chosen topical area: summary of the chosen area. The purpose is to describe why, how and for which purpose the area is used.
- **Method** applied in the study: how literature was selected and synthesized. For practical projects, how were tools and projects chosen, how were comparisons conducted.

[2] https://www.staff.lu.se/sites/staff.lu.se/files/2023-05/Guidelines-and-regulations-on-plagiarism-in-first-second-and-third-cycle-education-at-lund-university.pdf
[3] https://www.education.lu.se/digitala-verktyg/inspera-originality

- **Analysis**: analysis, critical thinking and future within the chosen area. For research projects, the purpose is to describe the benefits, drawbacks, what research the area needs in the future. For evaluation and practical projects, the work should be described here together with a shorter analysis.
- **Conclusion**: main conclusion in the report. The purpose is to discuss the main points in the report again.
- **Contribution statement**: A clear report what each project member contributed to, what was done jointly, and any use of AI tools.
- **References**: List of references with correct and consistent information. The report shall build on peer-reviewed research.

Another structure can be chosen after discussion with your project supervisor.

8 Presentation

The project will be presented at the project conference at the end of the course. Each project will be given 15 minutes to present their chosen area, including questions. The presentation should cover the main points of summary, analysis and conclusion. The project decides how to share presentation roles between them.

9 Practical Project Areas

9.1 Evaluate tools for software testing or review

Tools are important to implement software testing and reviews effectively in a software organization. However, although tools are needed, they do not solve the problems in the testing phase. Several tools exist and they must be evaluated before a software organization purchases one. Choose an area of interest, select a set of tools, and compare in a systematic way.

T. E. Vos, B. Marin, M. J. Escalona, and A. Marchetto. A methodological framework for evaluating software testing techniques and tools. In 12th International Conference on Quality Software. IEEE, Aug. 2012. DOI 10.1109/qsic.2012.16

P. Kuang, E. Söderberg, D. C. Niehorster, and M. Höst. Toward gaze-assisted developer tools. In 45th IEEE/ACM International Conference on Software Engineering: New Ideas and Emerging Results, Melbourne, Australia, pp. 49–54. IEEE, 2023. DOI 10.1109/ICSE-NIER58687.2023.00015

V. Garousi, N. Joy, Z. Jafarov, A. B. Kele¸s, S. De˘girmenci, E. Ozdemir, and R. Zarringhalami. AI-powered software testing tools: A systematic review and empirical assessment of their features and limitations, 2025. arXiv:2409.00411 [cs.SE] https://arxiv.org/abs/2409.00411

9.2 Evaluate test methods

Test methods may be more or less feasible for specific software systems. Therefore, it may be relevant, in addition to general evaluations, also evaluate test methods for specific systems or classes of systems. For a specific program, choose 2 or 3 test methods (one white-box and one black-box, or two methods of one type) and generate test cases from those, run the tests, log the outcome, and assess which method is best for the current system.

P. Runeson, C. Andersson, T. Thelin, A. Andrews, and T. Berling. What do we know about defect detection methods. IEEE Software, 23(3):82–+, 2006. DOI 10.1109/MS.2006.89

T. E. Vos, B. Marin, M. J. Escalona, and A. Marchetto. A methodological framework for evaluating software testing techniques and tools. In 12th International Conference on Quality Software. IEEE, Aug. 2012. DOI 10.1109/qsic.2012.16

9.3 Develop an automated test suite

The purpose of this project is to apply systematic approaches to an existing software system or an app. You may choose a project which you have developed on your own, or an open source project. Make sure it is limited enough to finish within the project scope, and complex enough to demonstrate the

approaches you have selected. To assess your automatic test suite, you may autogenerate an implementation of some functionality or to alter existing code by mutating its implementation.

L. Cruz, R. Abreu, R. and D. Lo, To the attention of mobile software developers: guess what, test your app!. Empir Software Eng 24, 2438–2468 (2019). DOI 10.1007/s10664-019-09701-0

S. Benli, A. Habash, A. Herrmann, T. Loftis and D. Simmonds, A Comparative Evaluation of Unit Testing Techniques on a Mobile Platform, 2012 Ninth International Conference on Information Technology - New Generations, Las Vegas, NV, 2012, pp. 263-268, DOI 10.1109/ITNG.2012.45

10 Research Project Areas

Below are some proposed project areas and an overview article related to each area. They are primarily aimed for research projects but may also be starting points for practical projects. If you would like to choose another area, describe the area to your project supervisor before starting the work.

10.1 How is testing performed for Agile processes?

Test-driven development (TDD) is often applied in agile software development. TDD is based on that test cases are first derived and used to specify how the system should work, as well as, used as test cases when the system is developed. In such development projects, the system behavior (or requirements) is discussed by testers and developers in close collaboration with their product owner. There are some interesting issues in this area. For example, what is the relationship between requirements and test cases?; can test cases be used to document requirements (thereby saving documentation effort)?; do such systems better satisfy what the customer wants in terms of functionality and quality?

E. Bjarnason, M. Unterkalmsteiner, M. Borg, and E. Engström. A multi-case study of agile requirements engineering and the use of test cases as requirements. Information & Software Technology, 77:61–79, 2016. DOI 10.1016/j.infsof.2016.03.008

A. Tosun, O. Dieste, D. Fucci, S. Vegas, B. Turhan, H. Erdogmus, A. Santos, M. Oivo, K. Toro, J. Järvinen, and N. Juristo. An industry experiment on the effects of test-driven development on external quality and productivity. Empirical Software Engineering, 22(6):2763–2805, 2017. DOI 10.1007/s10664-016-9490-0

D. Fucci, H. Erdogmus, B. Turhan, M. Oivo and N. Juristo, "A Dissection of the Test-Driven Development Process: Does It Really Matter to Test-First or to Test-Last?," in IEEE Transactions on Software Engineering, vol. 43, no. 7, pp. 597-614, 1 July 2017, DOI 10.1109/TSE.2016.2616877.

10.2 Can test cases be generated automatically from the code?

There is a class of testing methods, called search-based testing, that generate test criteria from the code. The approach is promising, but there are of course limitations to it. Recent advancements with LLMs have made significant contributions. Are they good enough? And what do they really test?

M. Harman and P. McMinn. A theoretical and empirical study of search-based testing: Local, global, and hybrid search. IEEE Transactions on Software Engineering, 36(2):226–247, 2010.

M. Harman, Y. Jia, and Y. Zhang. Achievements, open problems and challenges for search-based software testing. In 8th IEEE International Conference on Software Testing, Verification and Validation, ICST 2015, Graz, Austria, April 13-17, 2015, pages 1–12. IEEE Computer Society, 2015.

A. Arcuri. Restful API automated test case generation with EvoMaster. ACM Trans. Softw. Eng. Methodol., 28(1), 2019.

M. Schäfer, S. Nadi, A. Eghbali and F. Tip, An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation, in IEEE Transactions on Software Engineering, vol. 50, no. 1, pp. 85-105, Jan. 2024, doi: 10.1109/TSE.2023.3334955.

10.3 How to use LLMs to generate (good) tests?

Automating software testing activities has been a means to improve for a long time. With LLM advances, amazing progress has been observed in various fields. How can LLMs be used to generate efficient and effective test cases?

A. Fontes and G. Gay. The integration of machine learning into automated test generation: A systematic mapping study. Software Testing, Verification and Reliability, 33(4):e1845, 2023. DOI 10.1002/stvr.1845

S. Alagarsamy, C. Tantithamthavorn, and A. Aleti. A3test: Assertion-augmented automated test case generation. Inf. and Software Technology, 176:107565, 2024. DOI 10.1016/j.infsof.2024.107565

N. Alshahwan, J. Chheda, A. Finogenova, B. Gokkaya, M. Harman, I. Harper, A. Marginean, S. Sengupta, and E. Wang. Automated unit test improvement using large language models at meta. In Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, FSE, pages 185–196, 2024. ACM. DOI 10.1145/3663529.3663839

J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang. Software testing with large language models: Survey, landscape, and vision. IEEE Trans. Softw. Eng., 50(4):911–936, 2024. DOI 10.1109/TSE.2024.3368208

R. Santos, I. Santos, C. Magalhaes and R. de Souza Santos, Are We Testing or Being Tested? Exploring the Practical Applications of Large Language Models in Software Testing, 2024 IEEE Conference on Software Testing, Verification and Validation (ICST), Toronto, ON, Canada, 2024, pp. 353-360, doi: 10.1109/ICST60714.2024.00039.

10.4 How to test machine learning systems?

Machine learning is a buzzword of today that is applied to various application domains. However, very little is talked about how to verify system behavior, that is not specified but gradually learned by the system. Some researchers propose formal verification, but that is time consuming. What should we do?

Q. Bi and S. Chen, DeepCatch: A New Testing Framework for Convolutional Neural Networks, *3rd International Conference on Computer Science, Electronic Information Engineering and Intelligent Control Technology (CEI)*, Wuhan, China, 2023, pp. 83-87, doi: 10.1109/CEI60616.2023.10527909.

W. Ma, M. Papadakis, A. Tsakmalis, M. Cordy, and Y. L. Traon. Test selection for deep learning systems. ACM Trans. Softw. Eng. Methodol., 30(2), Jan. 2021. DOI 10.1145/3417330

J. M. Zhang, M. Harman, L. Ma and Y. Liu, Machine Learning Testing: Survey, Landscapes and Horizons, IEEE Transactions on Software Engineering, 48(1) 2022, doi: 10.1109/TSE.2019.2962027.

Q. Song, E. Engström, and P. Runeson. Concepts in testing of autonomous systems: Academic literature and industry practice. In J. Bosch, et al., editors, 1st Workshop on AI Engineering – Software Engineering for AI, ICSE 2021 Workshops, pages 74–81, IEEE, May 2021.

S. Li, J. Guo, J.-G. Lou, M. Fan, T. Liu, and D. Zhang. Testing machine learning systems in industry: an empirical study. In Proceedings of the 44th ICSE: Software Engineering in Practice, ICSE-SEIP '22, pages 263–272, 2022. ACM. DOI 10.1145/3510457.3513036

10.5 How to test interpretability in machine learning system?

Machine learning (ML) interpretability, as defined by Zhang et al., is "the degree to which an observer can understand the cause of a decision made by an ML system." Interpretability encompasses both inherently transparent models and explanations for black-box models. Interpretability is essential for understanding and assessing model behavior, enabling stakeholders to evaluate reliability, ensure accountability, and build justified trust in system outcomes. Furthermore, regulatory frameworks such as the EU General Data Protection Regulation (GDPR) and the EU Artificial Intelligence Act (AI Act) emphasize transparency, explainability, and accountability in automated decision-making systems.

The objectives of this project are twofold. First, to explore the key concepts of interpretability/ explainability in ML (e.g., ante-hoc vs. post-hoc explainability, model agnostic vs. model specific methods, local vs. global explanations). Second, to investigate practical approaches for evaluating interpretability in ML systems. This includes assessing functional properties such as fidelity, i.e., how accurately explanations reflect model behavior, and human interpretability, i.e., whether users can understand and act on explanations.

Longo, L., Brcic, M., Cabitza, F., Choi, J., Confalonieri, R., Del Ser, J., ... & Stumpf, S. (2024). Explainable Artificial Intelligence (XAI) 2.0: A manifesto of open challenges and interdisciplinary research directions. *Information Fusion*, *106*, 102301. DOI 10.1016/j.inffus.2024.102301

Angelov, P. P., Soares, E. A., Jiang, R., Arnold, N. I., & Atkinson, P. M. (2021). Explainable artificial intelligence: an analytical review. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, *11*(5), e1424. DOI 10.1002/widm.1424

Zhang, J. M., Harman, M., Ma, L., & Liu, Y. (2020). Machine learning testing: Survey, landscapes and horizons. *IEEE Transactions on Software Engineering*. DOI: 10.1109/TSE.2019.2962027

Doshi-Velez, F., and Kim, B. (2017). Towards a rigorous science of interpretable machine learning. arXiv preprint arXiv:1702.08608.

10.6 What are the minimal test practices needed in a start-up company?

When starting a new software company, the quality of the software is important to build trust among customers. However, startups generally have very little time to spend on testing. What is the minimal level of test practices needed to support a small software company?

C. Giardino, M. Unterkalmsteiner, N. Paternoster, T. Gorschek, and P. Abrahamsson. What do we know about software development in startups? Software, IEEE, 31(5):28–32, Sept 2014.

C. Giardino, N. Paternoster, M. Unterkalmsteiner, T. Gorschek and P. Abrahamsson, Software Development in Startup Companies: The Greenfield Startup Model, in IEEE Transactions on Software Engineering, vol. 42, no. 6, pp. 585-604, 2016, doi: 10.1109/TSE.2015.2509970.

Nguyen-Duc, A., Kemell, KK. & Abrahamsson, P. The entrepreneurial logic of startup software development: A study of 40 software startups. Empir Software Eng 26, 91 (2021). https://doi.org/10.1007/s10664-021-09987-z

10.7 Is it more efficient to generate test cases from a model?

Model-based testing is a popular term for generating test cases from various models, for example UML specifications. The advantage is that new test cases can be generated at low costs, once the model is defined. Maintenance of the test model may also be lower than maintaining individual test cases for evolutionary development.

A. Pretschner, W. Prenninger, S. Wagner, C. Kühnel, M. Baumhartner, B. Sostawa, R. Zolch, T. Stauner, One Evaluation of Model-Based Testing and its Automation, Proceedings. 27th International Conference on Software Engineering (ICSE) pp. 392–401, 2005.

Q. Malik, A. Jaaskelainen, H.Virtanen, M.Katara, F.Abbors, D.Truscan, and J.Lilius. Model-based testing using system vs. test models - what is the difference? 17th IEEE International Conference and Workshops on Engineering of Computer Based Systems, pp. 291–299, 2010.

T. Glock, B. Sillman, M. Kobold, S. Rebmann, E. Sax, Model-based validation and testing of industry 4.0 plants. Annual IEEE International Systems Conference (SysCon), Systems Conference (SysCon), page 1, 2018.

10.8 How do you test for security?

Security is an issue for more and more software systems, as they become more flexible, and handle more and more sensitive information, e.g. banking or health information. How can you test for security in a systematic and efficient way in the era of LLMs?

P. Morrison, R. Pandita, X. Xiao, R. Chillarege, and L. Williams. Are vulnerabilities discovered and resolved like other defects? Empirical Software Engineering, 23(3):1383–1421, 2018.

Ahsan, F., Anwer, F. A systematic literature review on software security testing using metaheuristics. Autom Softw Eng 31, 44 (2024). https://doi.org/10.1007/s10515-024-00433-0

X. Zhu, W. Zhou, Q. -L. Han, W. Ma, S. Wen and Y. Xiang, When Software Security Meets Large Language Models: A Survey, in IEEE/CAA Journal of Automatica Sinica, vol. 12, no. 2, pp. 317-334, February 2025, doi: 10.1109/JAS.2024.124971

10.9 Do static code analysis tools help improve software code quality?

Every static analyzer has a database of vulnerabilities to look for in code. Most of the products have the option of adding custom rules. Static analysis may be performed on modules or unfinished code, although the more complete the code, the more thorough and accurate the analysis can be. What static code analysis tools are available, and what are their strengths and weaknesses?

C. Sadowski, J. v. Gogh, C. Jaspan, E. Söderberg and C. Winter, Tricorder: Building a Program Analysis Ecosystem, 37th IEEE International Conference on Software Engineering (ICSE), IEEE, 2015, pp. 598-608. DOI 10.1109/ICSE.2015.76

B. Johnson, Y. Song, Y., E. Murphy-Hill. and R. Bowdidge, Why Don't Software Developers Use Static Analysis Tools to Find Bugs?, 35th International Conference on Software Engineering (ICSE). IEEE, 2013, pp. 672-681.

E. Söderberg, L. Church and M. Höst, Open Data-driven Usability Improvements of Static Code Analysis and its Challenges. In EASE'21: Evaluation and Assessment in Software Engineering (pp. 272-277) DOI 10.1145/3463274.3463808

10.10 How do you know that your test cases are effective?

Test cases are developed to check whether a software system is implemented correctly according to the requirements specification. However, if the test cases are badly chosen, they will not detect the failures. Mutation testing can be used to assess whether the test cases are feasible and effective.

M. Harman and Y. Jia, An analysis and survey of the development of mutation testing, IEEE Transactions on Software Engineering, 37 (5), 649-678, 2011

R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser. Are mutants a valid substitute for real faults in software testing? In Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE), pp 654–665, USA, 2014. ACM.

T. Pugh, C. Mina Lopez, S. Peacock, L. Deng, J. Dehlinger, and S. Chakraborty. Enhancing mutation testing efficiency: A comparative study of machine learning models for equivalent mutant identification. Software Testing, Verification and Reliability, 35(3–4), 2025. DOI 10.1002/stvr.70004

G. Wang, Q. Xu, L. Briand and K. Liu, Mutation-Guided Unit Test Generation With a Large Language Model, in IEEE Transactions on Software Engineering, vol. 52, no. 5, pp. 1657-1671, May 2026, doi: 10.1109/TSE.2026.3682975.

10.11 Flaky or non-deterministic tests

In continuous software engineering, test cases are executed repeatedly. Some test cases demonstrate a non-deterministic, or flaky, behavior in that they may pass or fail in subsequent executions on the same version of the code. Consequently, these tests are not trusted, even though there may be a real underlying fault. What constitutes a flakey test, and what can be done to mitigate the problems?

Q. Luo, F. Hariri, L. Eloussi, and D. Marinov. An empirical analysis of flaky tests. In Proceedings of the 22nd International Symposium on Foundations of Software Engineering, pages 643–653, 2014.

J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov. DeFlaker: Automatically detecting flaky tests. 40th International Conference on Software Engineering, pp. 433–444, 2018.

M. Eck, F. Palomba, M. Castelluccio, and A. Bacchelli. Understanding flaky tests: the developer's perspective. In Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 830–840, 2019.

10.12 How to test the energy consumption of software systems?

Software testing often focuses on the functional aspects of software systems, and less attention is devoted to quality attributes such as performance, maintainability, and efficiency. The energy consumption of software systems is particularly important in mobile devices, IoT systems, and AI systems.

The question is also important due to the implications of software systems in energy consumption that may result in GHG emissions. The idea of this project is to explore what are the concerns, challenges, and practices of testing energy consumption in software systems. Another approach could be to analyze the impact of metrics like testing coverage on the energy consumption of software systems.

P. Lago and I. Malavolta, A New Vision on Software Sustainability and Its Engineering, in IEEE Software, vol. 43, no. 1, pp. 119-123, Jan.-Feb. 2026, doi: 10.1109/MS.2025.3622804

García-Mireles, G. A., Moraga, M. Á., García, F., Calero, C., & Piattini, M. (2018). Interactions between environmental sustainability goals and software product quality: A mapping study. Information and Software Technology, 95, 108-129.

McIntosh, A., Hassan, S., & Hindle, A. (2019). What can Android mobile app developers do about the energy consumption of machine learning?. Empirical Software Engineering, 24(2), 562-601.

A. Bexell, L. G. Heander, E. Söderberg, S. Eldh, and P. Runeson. Exploring the performance of ML model size for classification in relation to energy consumption. Proceedings of the 26th International Conference on Product-Focused Software Process Improvement (PROFES), LNCS. Springer, 2025.

10.13 How to organize assertions in test cases?

Assertions is one essential component of a test case. An assertion is "a boolean expression that is placed at a certain point in a program to check its behaviour at runtime". The four-phase test design of setup, execution, validation and tear-down is recommended to structure test cases, with assertions embedded in the validation. What are the motivations for this structure and under which conditions is it challenged?

D. Bowes, T. Hall, J. Petric, T. Shippey and B. Turhan, How Good Are My Tests?, 2017 IEEE/ACM 8th Workshop on Emerging Trends in Software Metrics (WETSoM), Buenos Aires, Argentina, 2017, pp. 9-14, doi: 10.1109/WETSoM.2017.2.

C. Wei, L. Xiao, T. Yu, S. Wong, and A. Clune. How do developers structure unit test cases? An empirical analysis of the AAA pattern in open source projects. IEEE Transactions on Software Engineering, 51(4):1007–1038, Apr. 2025. http://dx.doi.org/10.1109/TSE.2025.3537337

M. Taromirad and P. Runeson. Assertions in software testing: Survey, landscape, and trends. International Journal on Software Tools for Technology Transfer, 27:117–135, 2025. https://doi.org/10.1007/s10009-025-00794-1